

Trabalho Prático Final

CCO-X: Variantes Computacionais do Método Constrained Constructive Optimization

Disciplina: Algoritmos e Programação Avançada
Prof. Dr. Rafael Alves Bonfim de Queiroz

2026.1

1 Objetivo Geral

O objetivo deste trabalho prático final é desenvolver, em linguagem C ou C++, uma versão funcional e estendida de um algoritmo inspirado no método CCO (*Constrained Constructive Optimization*) para geração de árvores arteriais.

O trabalho deve partir das implementações desenvolvidas nos exercícios MiniCCO-0 e MiniCCO-1, incorporando uma variante computacional inovadora, relacionada a algum dos seguintes temas:

- representação por grafos;
- função custo alternativa;
- otimização geométrica ou estrutural alternativa;
- uso de diagramas de Voronoi;
- quadtree ou octree;
- trifurcações ou multifurcações;
- paralelismo;
- domínio não convexo;
- geração 3D simplificada;
- análise topológica e estatística da árvore gerada.

2 Contexto

O método CCO clássico constrói uma árvore arterial de forma incremental. A cada passo, um novo ponto terminal é gerado dentro do domínio de perfusão, uma conexão temporária é criada com segmentos existentes, a nova bifurcação é otimizada geometricamente e a melhor conexão é escolhida segundo uma função custo.

No modelo clássico, a árvore arterial é representada como uma árvore binária de segmentos cilíndricos, com restrições de pressão, fluxo, raio, resistência hidráulica, ausência de interseções e distribuição espacial dos terminais.

Neste trabalho, cada grupo deverá implementar uma variante do CCO que preserve o núcleo básico do método, mas acrescente uma inovação algorítmica ou computacional.

3 Base Obrigatória

Todo grupo deverá obrigatoriamente implementar ou reaproveitar os seguintes componentes:

1. representação de pontos, segmentos e nós arteriais;
2. árvore binária ou estrutura equivalente;
3. geração de pontos terminais no domínio;
4. teste de pertencimento ao domínio;
5. cálculo de distância ponto-ponto e ponto-segmento;
6. teste de interseção ou colisão;
7. seleção de conexões candidatas;
8. escala dos raios segundo lei de bifurcação;
9. cálculo de comprimento, resistência e volume;
10. função custo baseada em volume intravascular;
11. exportação dos resultados em `.csv`, `.txt` ou `.vtk`;
12. visualização em Python, preferencialmente usando PyVista.

4 Função Custo Básica

A função custo mínima obrigatória será o volume intravascular total:

$$J = V_{total} = \sum_{j=1}^{N_{seg}} \pi r_j^2 l_j,$$

onde:

- r_j é o raio do segmento j ;
- l_j é o comprimento do segmento j ;
- N_{seg} é o número total de segmentos.

Opcionalmente, o grupo pode propor uma função custo composta:

$$J = \alpha V_{total} + \beta L_{total} + \delta P_{\theta} + \eta P_{inter},$$

onde:

- V_{total} é o volume intravascular;
- L_{total} é o comprimento total da árvore;
- P_{θ} é uma penalização angular;
- P_{inter} é uma penalização por interseção, proximidade ou violação geométrica;
- $\alpha, \beta, \delta, \eta$ são pesos definidos pelo grupo.

5 Lei de Bifurcação

Em cada bifurcação, os raios deverão satisfazer:

$$r_p^{\gamma} = r_1^{\gamma} + r_2^{\gamma},$$

onde:

- r_p é o raio do segmento pai;
- r_1 e r_2 são os raios dos segmentos filhos;
- γ é o expoente de bifurcação.

Valores recomendados:

$$\gamma = 3.0$$

ou, para comparação:

$$\gamma \in \{2.55, 2.7, 3.0\}.$$

6 Parâmetros Físicos Recomendados

Utilize como referência:

$$\begin{aligned} Q_{perf} &= 8.33 \times 10^{-6} \text{ m}^3/\text{s}, \\ p_{perf} &= 1.33 \times 10^4 \text{ Pa}, \\ p_{term} &= 7.98 \times 10^3 \text{ Pa}, \\ \mu &= 3.6 \times 10^{-3} \text{ Pa} \cdot \text{s}. \end{aligned}$$

Para fluxos terminais iguais:

$$Q_{term} = \frac{Q_{perf}}{N_{term}}.$$

7 Temas Possíveis

Cada grupo deverá escolher um dos projetos abaixo.

7.1 Projeto 1 – GraphCCO

Representar a árvore arterial como um grafo ponderado.

Inovação: usar lista de adjacência para analisar topologia, caminhos, níveis, profundidade, grau dos nós e ordem de Strahler.

Obrigatório:

- converter a árvore CCO em grafo;
- implementar BFS e DFS;
- calcular caminho da raiz até cada terminal;
- calcular profundidade máxima;
- calcular comprimento dos caminhos terminais;
- exportar grafo para visualização.

7.2 Projeto 2 – CostCCO

Implementar e comparar diferentes funções custo.

Inovação: estudar como a escolha da função custo altera a morfologia da árvore.

Funções sugeridas:

$$J_1 = V_{total},$$

$$J_2 = L_{total},$$

$$J_3 = \alpha V_{total} + \beta L_{total},$$

$$J_4 = \alpha V_{total} + \beta P_{\theta}.$$

Obrigatório:

- comparar pelo menos três funções custo;
- gerar gráficos comparativos;
- discutir impacto no volume, comprimento, raio médio e forma da árvore.

7.3 Projeto 3 – GeoCCO

Implementar uma estratégia alternativa de otimização geométrica da bifurcação.

Inovação: comparar busca em grade, busca gulosa local e busca por refinamento sucessivo.

Obrigatório:

- otimizar a bifurcação em uma região triangular;
- comparar pelo menos duas estratégias;
- medir custo final e tempo de execução;
- discutir qualidade da solução.

7.4 Projeto 4 – VoronCCO

Usar diagramas de Voronoi ou uma aproximação discreta para distribuir terminais.

Inovação: substituir a geração puramente aleatória por uma geração espacialmente mais uniforme.

Obrigatório:

- gerar sementes no domínio;
- associar pontos do domínio à semente mais próxima;
- gerar terminais guiados por regiões;
- comparar com geração aleatória;
- medir uniformidade espacial dos terminais.

7.5 Projeto 5 – FastCCO

Usar uma estrutura espacial, como quadtree em 2D ou octree em 3D.

Inovação: reduzir o custo da busca por segmentos próximos e testes de interseção.

Obrigatório:

- implementar quadtree ou octree;
- inserir segmentos ou pontos na estrutura;
- buscar vizinhos próximos;
- comparar tempo de execução com busca exaustiva;
- discutir complexidade.

7.6 Projeto 6 – TriCCO

Permitir trifurcações ou multifurcações controladas.

Inovação: generalizar o modelo clássico binário.

Obrigatório:

- modificar a estrutura de dados para permitir mais de dois filhos;
- propor uma lei de escala para múltiplos filhos:

$$r_p^\gamma = \sum_{k=1}^m r_k^\gamma;$$

- comparar árvore binária com árvore contendo trifurcações;
- discutir vantagens e limitações fisiológicas.

7.7 Projeto 7 – ParallelCCO

Paralelizar a avaliação de conexões candidatas ou pontos de otimização geométrica.

Inovação: explorar paralelismo em C/C++.

Obrigatório:

- paralelizar a avaliação dos candidatos;
- medir tempo sequencial e paralelo;
- calcular speedup:

$$S = \frac{T_{seq}}{T_{par}};$$

- discutir gargalos.

7.8 Projeto 8 – DomainCCO

Implementar domínio não circular, não convexo ou com obstáculos internos.

Inovação: aproximar situações anatômicas mais complexas.

Obrigatório:

- implementar teste de domínio geral;
- impedir crescimento dentro de obstáculos;
- adaptar geração de pontos terminais;
- comparar domínio circular com domínio irregular.

7.9 Projeto 9 – CCO3D-Simplificado

Estender a implementação para 3D.

Inovação: gerar árvores dentro de uma esfera ou caixa tridimensional.

Obrigatório:

- representar pontos em 3D;
- gerar terminais em esfera;
- calcular distâncias 3D;
- exportar para VTK;
- visualizar em PyVista.

8 Entrada do Programa

Cada grupo deverá definir os parâmetros de entrada, mas recomenda-se o seguinte formato:

```
./trabalho Nterm R gamma modo seed
```

onde:

- **Nterm**: número de terminais;
- **R**: raio ou dimensão principal do domínio;
- **gamma**: expoente de bifurcação;
- **modo**: variante implementada;
- **seed**: semente do gerador aleatório.

Exemplo:

```
./trabalho 100 0.05 3.0 cost 123
```

9 Saídas Obrigatórias

O programa deverá gerar:

- arquivo com os segmentos da árvore;
- arquivo com métricas globais;
- arquivo de log da execução;
- visualização da árvore;
- relatório técnico.

O arquivo de segmentos deve conter, no mínimo:

```
id, pai, x0, y0, z0, x1, y1, z1, raio, comprimento,
fluxo, resistencia, volume
```

O arquivo de métricas deve conter:

```
Nterm, Nseg, comprimento_total, volume_total, raio_raiz,
raio_medio, profundidade_maxima, tempo_execucao
```

10 Visualização

A visualização deverá ser feita em Python, preferencialmente usando PyVista.

A árvore deve ser visualizada com:

- segmentos como linhas ou tubos;
- espessura proporcional ao raio;
- raiz destacada;
- terminais destacados;
- se possível, coloração por nível, raio ou fluxo.

11 Experimentos Mínimos

Cada grupo deverá executar pelo menos:

Teste	N_{term}	γ	Sementes
1	20	3.0	3
2	50	3.0	3
3	100	3.0	3
4	100	2.7	3
5	100	2.55	3

Para cada teste, calcular média e desvio padrão das métricas principais.

12 Métricas a Reportar

O relatório deverá apresentar:

- comprimento total;
- volume intravascular total;
- raio da raiz;
- raio médio;

- profundidade máxima;
- número de bifurcações;
- número de terminais;
- tempo de execução;
- número de conexões testadas;
- número de conexões rejeitadas.

Projetos baseados em grafos deverão também reportar:

- grau médio;
- distribuição de profundidade;
- caminhos raiz-terminal;
- ordem de Strahler, se implementada.

13 Relatório Técnico

O relatório deverá conter entre 6 e 10 páginas, com:

1. introdução;
2. descrição do método CCO;
3. descrição da variante implementada;
4. estruturas de dados utilizadas;
5. algoritmos implementados;
6. função custo;
7. experimentos;
8. resultados;
9. análise de complexidade;
10. conclusão.

14 Análise de Complexidade

A análise deve discutir o custo de:

- geração dos terminais;
- busca por candidatos;
- teste de interseção;
- otimização geométrica;
- atualização de raios;
- exportação e visualização.

Considere:

- N : número de segmentos;
- K : número de candidatos avaliados;
- M : resolução da busca geométrica;
- S : número de sementes ou execuções independentes.

Por exemplo, uma busca exaustiva simples pode ter custo:

$$O(KM^2N)$$

quando cada candidato exige uma busca em grade e recalcula propriedades da árvore.

15 Apresentação Final

Cada grupo deverá apresentar o trabalho em 10 a 15 minutos, contendo:

- problema escolhido;
- variante implementada;
- principais algoritmos;
- demonstração da árvore gerada;
- resultados numéricos;
- limitações;
- possíveis extensões.

16 Entregáveis

Cada grupo deverá entregar:

- código-fonte completo;
- instruções de compilação;
- arquivos de entrada e saída;
- script de visualização em Python;
- relatório técnico em PDF;
- slides da apresentação;
- pelo menos uma imagem ou vídeo curto da árvore gerada.

17 Critérios de Avaliação

Item	Peso
Implementação do núcleo CCO	20%
Implementação da variante inovadora	25%
Correção algorítmica e geométrica	15%
Experimentos e análise numérica	15%
Visualização e organização das saídas	10%
Relatório técnico	10%
Apresentação final	5%

18 Observações Importantes

- O trabalho não precisa reproduzir integralmente os artigos clássicos do CCO.
- O objetivo é implementar um núcleo funcional e uma extensão computacional relevante.
- A variante escolhida deve estar claramente descrita e justificada.
- O código deve ser organizado, modular e comentado.
- Resultados negativos ou limitações também devem ser discutidos.

19 Sugestão de Organização do Código

Sugere-se a seguinte organização:

```
src/  
main.cpp  
point.h  
geometry.h  
geometry.cpp  
tree.h  
tree.cpp  
cco.h  
cco.cpp  
cost.h  
cost.cpp  
export.h  
export.cpp  
  
python/  
visualize.py  
  
results/  
outputs/  
figures/
```

20 Síntese

Ao final do trabalho, espera-se que cada grupo tenha desenvolvido uma implementação capaz de:

- gerar uma árvore arterial simplificada;
- aplicar restrições geométricas;
- ajustar raios e fluxos;
- calcular volume intravascular;
- otimizar conexões ou bifurcações;
- implementar uma variante inovadora;
- visualizar e analisar os resultados.